



Plateforme d'orientation pour étudiants africains

Documentation Technique Complète

Version 1.0 · Juin 2026

Architecture · Base de données · API · Frontend · Sécurité · Déploiement

Confidentiel — usage interne

Table des matières

1. Présentation générale	3
1.1 Vision et objectifs	3
1.2 Publics cibles	3
1.3 Fonctionnalités principales	3
2. Architecture globale	4
2.1 Vue d'ensemble	4
2.2 Stack technologique	4
2.3 Services externes	4
3. Structure des fichiers	5
4. Base de données	6
4.1 Schéma des tables	6
4.2 Diagramme entité-relation	8
5. API Backend — Routes	9
5.1 Authentification /api/auth	9
5.2 Utilisateur /api/user	9
5.3 Profil /api/profile	9
5.4 Documents /api/documents	10
5.5 Analyse IA /api/analysis	10
5.6 Bourses /api/bourses	10
5.7 Universités /api/universities	10
5.8 Logement /api/logement	10
5.9 Lettre de motivation /api/lm	10
5.10 CV /api/cv	10
5.11 Admin /api/admin	11
6. Frontend — Pages et composants	12

7. Flux d'authentification	14
7.1 Inscription / Email	14
7.2 Google OAuth2	14
7.3 Téléphone OTP (Twilio)	14
7.4 Double authentification (2FA)	14
8. Gestion des documents	15
9. Moteur d'analyse IA	16
10. Diagrammes UML	17
10.1 Diagramme de cas d'utilisation	17
10.2 Diagramme de séquence — Analyse IA	18
11. Sécurité	19
12. Forces et faiblesses	20
13. Recommandations techniques	21
14. Installation et déploiement	22
15. Guide de maintenance	23

1. Présentation générale

1.1 Vision et objectifs

Wekili (du swahili : « avocat, défenseur ») est une plateforme web full-stack destinée à accompagner les étudiants africains francophones dans leur projet d'études à l'international. La plateforme centralise la recherche de bourses, l'évaluation du dossier académique par intelligence artificielle, la gestion des documents officiels, le suivi des candidatures universitaires, la préparation des lettres de motivation et CV, ainsi que la recherche de logement.

1.2 Publics cibles

Étudiants africains

Profils Licence, Master ou Doctorat souhaitant poursuivre leurs études en France, Canada, Belgique, Allemagne ou Royaume-Uni.

Administrateurs

Équipe Wekili gérant le catalogue de bourses et universités, supervisant les utilisateurs et les analyses.

1.3 Fonctionnalités principales

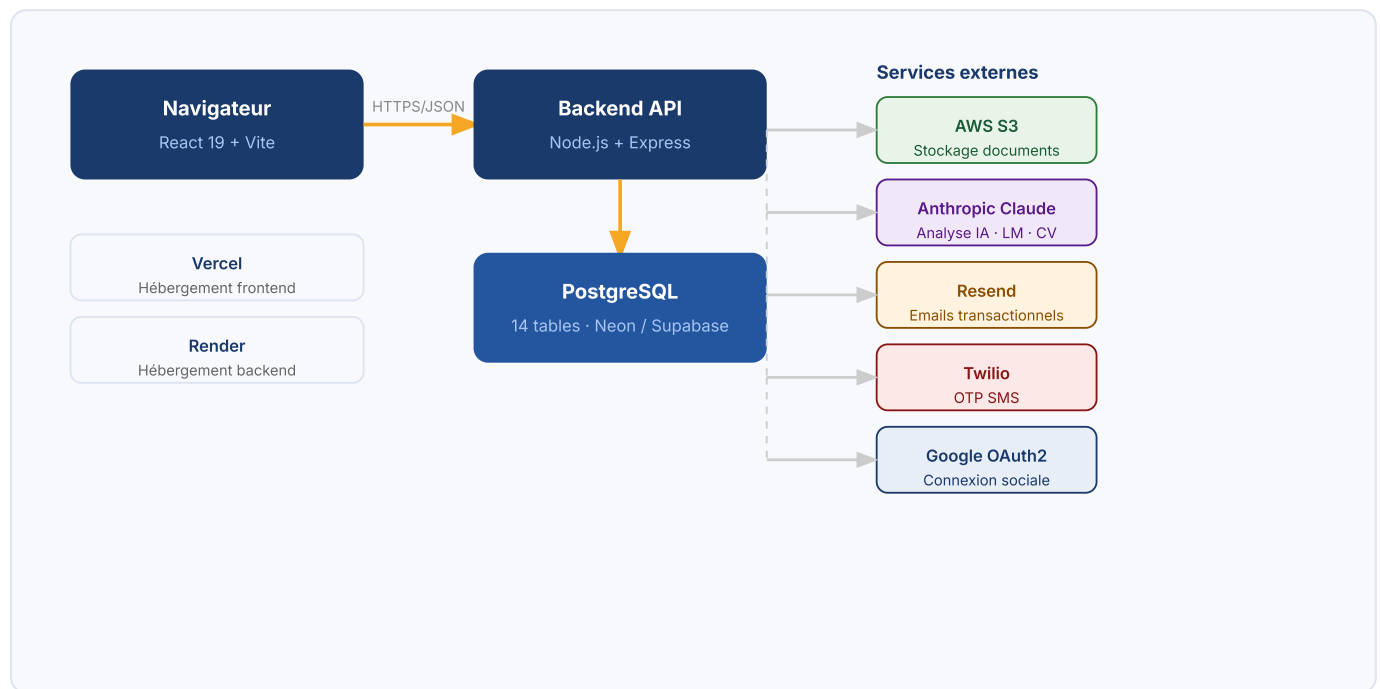
Module	Fonctionnalité	Technologie clé
Authentification	Email/mot de passe, Google OAuth2, OTP SMS, 2FA TOTP	JWT · Google Auth Library · Twilio · Resend
Profil académique	Saisie des données académiques, langues, pays cibles, budget	PostgreSQL · React Forms
Documents	Upload sécurisé (PDF/image), PIN, accès OTP, journal d'activité	AWS S3 · Multer · bcryptjs
Analyse IA	Score d'éligibilité, forces/faiblesses, programmes recommandés	Anthropic Claude API
Bourses	Catalogue enrichi, matching par profil, score d'éligibilité	PostgreSQL JSONB
Universités	15 universités avec critères détaillés, suivi de candidature	PostgreSQL · React
Lettre de motivation	Génération et correction IA, versioning	Anthropic Claude API
CV	Extraction PDF, analyse, correction adaptée au pays cible	pdf-parse v2 · Claude API

Logement	Plateformes par pays, aides financières, checklist, guide IA	React · Claude API
Administration	Gestion utilisateurs, rôles, bourses, universités, statistiques	ProtectedAdminRoute · RBAC

2. Architecture globale

2.1 Vue d'ensemble

Wekili suit une architecture **SPA + API REST** découplée. Le frontend React communique exclusivement avec le backend Express via des appels HTTP JSON. Aucun rendu côté serveur (SSR) n'est utilisé.



2.2 Stack technologique

Frontend

Outil	Version	Rôle
React	19.x	UI composants
Vite	6.x	Bundler / Dev server
React Router	7.x	Routage SPA
Tailwind CSS	3.x	Styles utilitaires
flag-icons	7.x	Drapeaux pays

Backend

Outil	Version	Rôle
Node.js	≥18	Runtime
Express	5.x	Framework HTTP
pg	8.x	Client PostgreSQL
jsonwebtoken	9.x	Tokens JWT
bcryptjs	3.x	Hachage mots de passe
multer	2.x	Upload fichiers
helmet	8.x	Headers sécurité

express-rate-limit	8.x	Rate limiting
--------------------	-----	---------------

3. Structure des fichiers

Backend

```

backend/
├─ server.js           # Point d'entrée
Express
├─ package.json
├─ config/
│   └─ database.js     # Pool PostgreSQL
├─ database/
│   └─ setup_all.js    # Migrations
idempotentes
├─ middleware/
│   └─ auth.js         # Vérification JWT
(user)
│   └─ authAdmin.js    # Vérification JWT
(admin)
├─ routes/
│   └─ auth.js
│   └─ user.js
│   └─ profile.js
│   └─ documents.js
│   └─ analysis.js
│   └─ bourses.js
│   └─ universities.js
│   └─ logement.js
│   └─ lm.js
│   └─ cv.js
│   └─ admin.js
└─ controllers/
    └─ authController.js
    └─ userController.js
    └─ profileController.js
    └─ documentsController.js
    └─ analysisController.js
    └─ boursesController.js
    └─ universitiesController.js
    └─ logementController.js
    └─ lmController.js
    └─ cvController.js
    └─ adminController.js
  
```

Frontend

```

frontend/
├─ index.html
├─ vite.config.js
├─ tailwind.config.js
├─ src/
│   └─ App.jsx         # Routeur principal
│   └─ App.css
│   └─ main.jsx
│   └─ utils/
│       └─ auth.js     # getToken / getUser
│   └─ services/
│       └─ api.js      # Appels API
centralisés
│   └─ components/
│       └─ Navbar.jsx
│       └─ Toast.jsx
│   └─ pages/
│       └─ Home.jsx
│       └─ Login.jsx
│       └─ Register.jsx
│       └─ Dashboard.jsx
│       └─ Profile.jsx
│       └─ Documents.jsx
│       └─ Analysis.jsx
│       └─ Bourses.jsx
│       └─ Universities.jsx
│       └─ Logement.jsx
│       └─ Settings.jsx
│       └─ VerifyEmail.jsx
│       └─ ForgotPassword.jsx
│       └─ PrivacyPolicy.jsx
│       └─ TermsOfService.jsx
│       └─ LegalNotice.jsx
│   └─ admin/
│       └─ AdminLayout.jsx
│       └─ AdminOverview.jsx
│       └─ AdminUsers.jsx
│       └─ AdminBourses.jsx
│       └─ AdminUniversites.jsx
  
```


4. Base de données

PostgreSQL est utilisé avec le driver `pg`. Toutes les migrations sont regroupées dans `setup_all.js` et exécutées au démarrage du serveur dans une transaction unique. Les instructions `CREATE TABLE IF NOT EXISTS` et `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` garantissent l'idempotence.

4.1 Schéma des tables

users

Colonne	Type	Contrainte	Description
<code>id</code>	UUID	PK · DEFAULT <code>gen_random_uuid()</code>	Identifiant unique
<code>email</code>	VARCHAR(255)	UNIQUE	Email de connexion
<code>password</code>	VARCHAR(255)		Hash bcrypt (null si OAuth)
<code>nom</code>	VARCHAR(100)		Nom de famille
<code>prenom</code>	VARCHAR(100)		Prénom
<code>pays</code>	VARCHAR(100)		Pays d'origine
<code>google_id</code>	VARCHAR(255)	UNIQUE	ID Google OAuth
<code>phone</code>	VARCHAR(30)	UNIQUE	Numéro de téléphone
<code>avatar_url</code>	TEXT		URL photo de profil
<code>auth_method</code>	VARCHAR(20)	DEFAULT 'email'	email / google / phone
<code>two_fa_enabled</code>	BOOLEAN	DEFAULT FALSE	2FA TOTP activé
<code>email_verified</code>	BOOLEAN	DEFAULT FALSE	Email confirmé
<code>role</code>	VARCHAR(20)	DEFAULT 'user'	user / admin / superadmin
<code>created_at</code>	TIMESTAMPTZ	DEFAULT NOW()	Date de création

profiles

Lié à `users` en relation 1-1 via `user_id`. Contient les données académiques : nationalité, niveau d'études, domaine, établissement, moyenne, langues maîtrisées, pays cibles, budget, motivation, expérience professionnelle.

documents

Colonne	Type	Description
<code>id</code>	UUID PK	Identifiant document

<code>user_id</code>	UUID FK → users	Propriétaire
<code>type</code>	VARCHAR(100)	releve_notes / diplome / passeport / cv / lettre_motivation / etc.
<code>nom_fichier</code>	VARCHAR(500)	Nom original du fichier
<code>url_s3</code>	TEXT	Clé S3 du fichier
<code>taille</code>	INTEGER	Taille en octets
<code>statut</code>	VARCHAR(50)	en_attente / valide / rejete

analyses

Stocke les résultats des analyses IA : `score_global` (INTEGER), `synthese` (TEXT), `points_forts`, `axes_amelioration`, `forces`, `faiblesses`, `recommandations`, `programmes_recommandes`, `estimation_chances` (tous JSONB).

scholarships (bourses)

Catalogue de 9 bourses enrichies : nom, organisme, pays, niveau, domaine, montant, deadline, critères (JSONB), avantages (TEXT), documents requis (TEXT[]), nationalités éligibles (TEXT[]), âge max, nombre de places, type de financement.

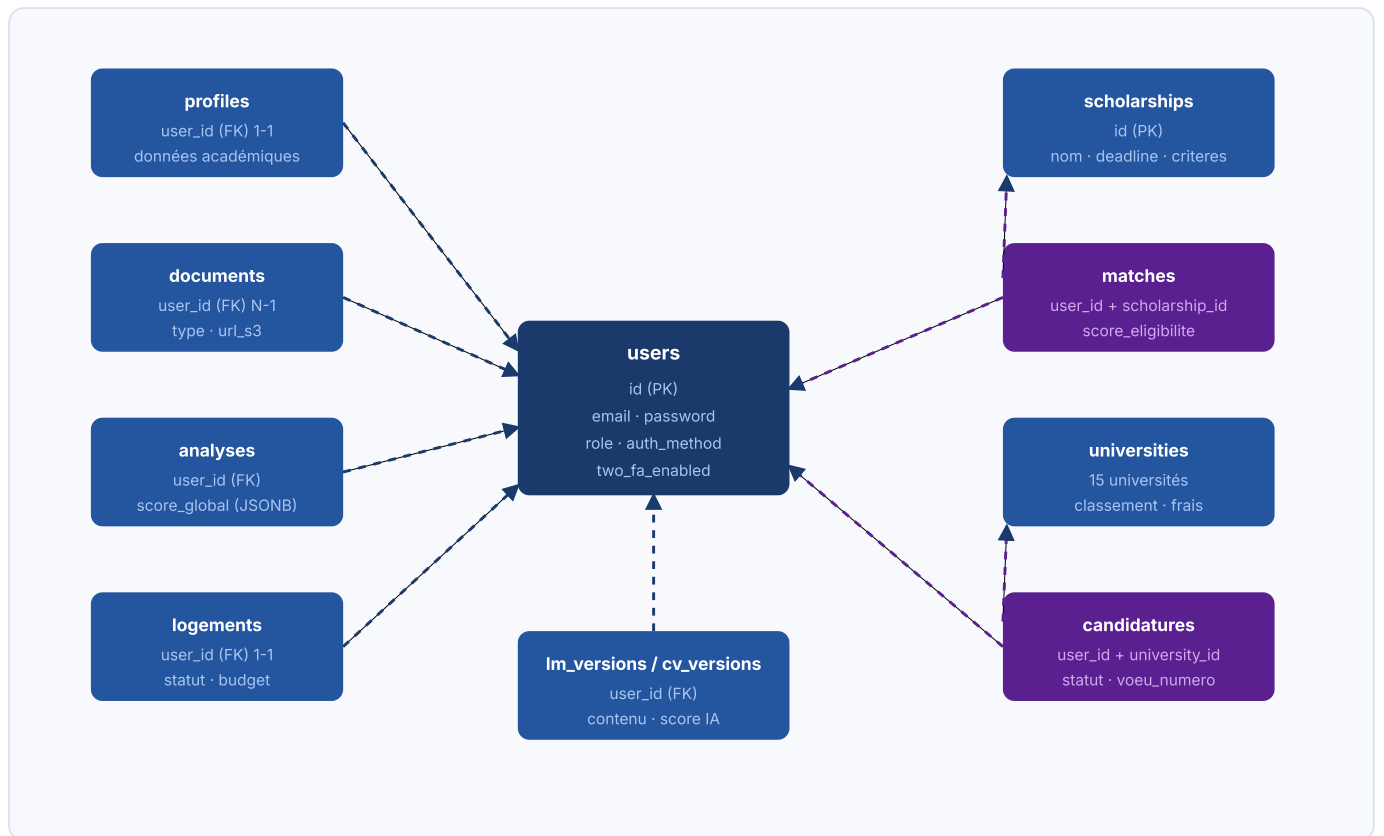
universities

15 universités pré-chargées couvrant France (5), Canada (3), Belgique (2), Allemagne (2), Royaume-Uni (2). Champs : classement mondial/national, taux d'admission, frais de scolarité, plateformes de candidature, dates d'ouverture/clôture, documents requis (TEXT[]).

Autres tables

Table	Rôle
<code>document_logs</code>	Journal de toutes les actions sur documents (upload, suppression, accès)
<code>login_sessions</code>	Historique des connexions : IP, user-agent, device_hash
<code>matches</code>	Score d'éligibilité utilisateur × bourse (UNIQUE user_id + scholarship_id)
<code>university_matches</code>	Score d'admission utilisateur × université
<code>candidatures</code>	Suivi des candidatures : statut, numéro de vœu, notes, date de soumission
<code>logements</code>	Préférences et statut de recherche logement par utilisateur (1-1)
<code>lm_versions</code>	Versions des lettres de motivation : contenu, score IA, version corrigée
<code>cv_versions</code>	Versions CV : contenu, pays cible, corrections IA, sections manquantes

4.2 Diagramme entité-relation simplifié



5. API Backend — Routes

Le serveur Express écoute sur `PORT=5000` (configurable). Rate limiting global : **200 req / 15 min**. CORS autorisé sur `wekili.vercel.app`, `localhost:5173` et `localhost:3000`. Les routes protégées vérifient le header `Authorization: Bearer <JWT>` via le middleware `auth.js`.

5.1 /api/auth — Authentification

Rate limit spécifique : 10 tentatives / 15 min (`skipSuccessfulRequests: true`)

Méthode	Endpoint	Description
POST	/register	Inscription email + envoi email de vérification (Resend)
POST	/login	Connexion email / mot de passe → JWT
POST	/google	Connexion Google OAuth2 (token Google → JWT)
POST	/facebook	Connexion Facebook OAuth
POST	/phone/send-otp	Envoi OTP SMS via Twilio
POST	/phone/verify	Vérification OTP → JWT
POST	/2fa-verify	Vérification code TOTP 2FA
POST	/verify-email	Confirmation du lien email
POST	/resend-verify	Renvoi de l'email de vérification
POST	/forgot-password	Envoi lien de réinitialisation (Resend)
POST	/reset-password	Réinitialisation du mot de passe

5.2 /api/user — Compte utilisateur

Méthode	Endpoint	Description
PATCH	/	Mise à jour nom, prénom, pays, avatar
POST	/change-password	Changement de mot de passe (ancien requis)
DELETE	/	Suppression du compte
GET	/sessions	Historique des connexions (IP, device, date)
POST	/2fa/enable	Activation 2FA → génération secret TOTP

POST	/2fa/confirm	Confirmation 2FA avec premier code
POST	/2fa/disable	Désactivation 2FA

5.3 /api/documents — Gestion documentaire

Méthode	Endpoint	Description
GET	/	Liste des documents de l'utilisateur
POST	/upload	Upload vers S3 (max 10 MB, PDF/JPG/PNG/DOC/DOCX)
DELETE	/:id	Suppression S3 + DB
GET	/logs	Journal d'activité documents
GET	/pin/check	Vérification existence PIN
POST	/pin/create	Création PIN (haché bcrypt)
POST	/pin/verify	Vérification PIN pour accès documents
POST	/pin/reset/request	Demande réinitialisation PIN (email OTP)
POST	/pin/reset/confirm	Confirmation réinitialisation PIN

5.4 /api/analysis — Analyse IA

Méthode	Endpoint	Description
POST	/launch	Lance l'analyse IA du profil → score + recommandations
GET	/latest	Dernière analyse de l'utilisateur
GET	/all	Historique de toutes les analyses

5.5 Routes supplémentaires

Préfixe	Endpoints	Description
/api/bourses	GET /public · GET / · GET /:id	Catalogue bourses (public + authentifié)
/api/universities	GET / · GET /:id · POST / (candidature) · PATCH /:id	Universités + gestion candidatures
/api/logement	GET / · PUT / · POST /guide-ia	Préférences logement + guide IA
/api/lm	GET / · POST /generate · POST /correct · POST /save	Lettre de motivation IA

<code>/api/cv</code>	GET / · POST /correct · POST /save · POST /upload	CV — extraction PDF + correction IA
<code>/api/profile</code>	GET / · PUT /	Profil académique complet

5.6 /api/admin — Administration (role admin/superadmin)

Middleware `authAdmin.js` : vérifie JWT + `role IN ('admin', 'superadmin')` . Retourne 403 sinon.

Méthode	Endpoint	Description
GET	<code>/stats</code>	Statistiques globales (utilisateurs, analyses, documents)
GET	<code>/users</code>	Liste paginée des utilisateurs
GET	<code>/users/:id</code>	Détail utilisateur (profil + analyses + documents)
PATCH	<code>/users/:id/role</code>	Modification du rôle utilisateur
DELETE	<code>/users/:id</code>	Suppression compte utilisateur
GET	<code>/bourses</code>	Toutes les bourses (admin)
POST	<code>/bourses</code>	Création bourse
PATCH	<code>/bourses/:id</code>	Modification bourse
DELETE	<code>/bourses/:id</code>	Suppression bourse
GET	<code>/universities</code>	Toutes les universités
POST	<code>/universities</code>	Création université
PATCH	<code>/universities/:id</code>	Modification université
DELETE	<code>/universities/:id</code>	Suppression université

6. Frontend — Pages et composants

6.1 Routage et protection

Le routage est géré par **React Router v7**. Trois gardes de route sont implémentés :

Composant	Comportement
<code>ProtectedRoute</code>	Redirige vers <code>/login</code> si aucun JWT en localStorage. Mémoire la destination pour y revenir après connexion.
<code>PublicOnlyRoute</code>	Redirige vers <code>/dashboard</code> si déjà connecté (évite l'accès à <code>/login</code> et <code>/register</code>).
<code>ProtectedAdminRoute</code>	Vérifie JWT + <code>role IN ['admin', 'superadmin']</code> . Redirige vers <code>/dashboard</code> sinon.

6.2 Pages publiques

Page	Route	Description
Home	<code>/</code>	Landing page : présentation, étapes de préparation, niveaux d'étude, bourses vedettes dynamiques, statistiques, témoignages.
Login	<code>/login</code>	Connexion email, Google, téléphone. Gestion 2FA. Lien mot de passe oublié.
Register	<code>/register</code>	Inscription multi-étapes avec présentation des fonctionnalités. Connexion Google disponible.
VerifyEmail	<code>/verify-email</code>	Confirmation du lien reçu par email.
ForgotPassword	<code>/forgot-password</code>	Envoi email de réinitialisation + formulaire nouveau mot de passe.
PrivacyPolicy	<code>/confidentialite</code>	Politique de confidentialité RGPD.
TermsOfService	<code>/cgu</code>	Conditions générales d'utilisation.
LegalNotice	<code>/mentions-legales</code>	Mentions légales.

6.3 Pages protégées (utilisateur)

Page	Route	Fonctionnalités clés
Dashboard	<code>/dashboard</code>	Vue synthèse : score IA, compteur bourses, profil %, documents. Bourses urgentes. Accès rapide.

Profile	/profile	Formulaire académique complet : nationalité, niveau, domaine, moyenne, langues (2), pays cibles, budget, motivation, expérience.
Documents	/documents	Upload de 6 types de documents, système PIN de sécurité, journal d'activité, visualisation S3.
Analysis	/analysis	Score IA, synthèse, forces/faiblesses, programmes recommandés. Génération + correction IA de LM. Analyse + correction CV adaptée au pays cible.
Bourses	/bourses	Catalogue filtrable (pays, niveau, domaine, financement). Score d'éligibilité par bourse. Détail modal avec critères et documents.
Universities	/universities	15 universités filtrables. Score d'admission, suivi de candidature (statut + numéro de vœu), deadlines.
Logement	/logement	Plateformes par pays, aides financières, estimation budget par ville, checklist en groupes, guide IA.
Settings	/settings	Modification compte, changement mot de passe, gestion 2FA, historique sessions, suppression compte.

6.4 Pages admin

Page	Route	Fonctionnalités
AdminOverview	/admin	Statistiques globales, graphiques activité, métriques utilisateurs
AdminUsers	/admin/users	Liste utilisateurs, recherche, détail (profil + analyses), modification rôle, suppression
AdminBourses	/admin/bourses	CRUD complet du catalogue de bourses
AdminUniversities	/admin/universities	CRUD complet du catalogue d'universités

6.5 Composants partagés

Navbar

Barre de navigation adaptative (mobile/desktop). Liens contextuels selon l'état de connexion. Lien vers /admin si rôle admin.

Toast

Système de notifications toast (succès, erreur, info). Utilisé dans toutes les pages pour les retours actions API.

7. Flux d'authentification

7.1 Inscription par email



7.2 Connexion Google OAuth2

Le frontend reçoit le `credential` Google (ID Token) via Google Identity Services. Il l'envoie à `POST /api/auth/google`. Le backend vérifie le token avec `google-auth-library` (`OAuth2Client.verifyIdToken`), extrait l'email, prénom, nom, avatar. Si l'utilisateur n'existe pas, il est créé avec `auth_method='google'` et `email_verified=true`. Un JWT Wekili est retourné.

7.3 Connexion par téléphone (Twilio OTP)

1. `POST /phone/send-otp` : génération d'un code OTP, envoi SMS via Twilio Verify
2. `POST /phone/verify` : vérification du code → création/récupération compte → JWT

7.4 Double authentification (2FA TOTP)

Lors de l'activation via `POST /user/2fa/enable`, le backend génère un secret TOTP et retourne un QR code compatible Google Authenticator. L'utilisateur confirme avec un premier code (`/2fa/confirm`). Lors des connexions suivantes, si `two_fa_enabled=true`, le login retourne un token temporaire et redirige vers la vérification `POST /auth/2fa-verify`.

7.5 Payload JWT

```

{
  "id": "uuid-utilisateur",
  "email": "user@example.com",
  "role": "user" | "admin" | "superadmin"
}
  
```

Le token est stocké dans `localStorage` et transmis via le header `Authorization: Bearer <token>`. Le middleware `auth.js` décode le token et injecte `req.user`.

8. Gestion des documents

8.1 Pipeline d'upload

Le fichier transite en mémoire via `multer.memoryStorage()` (max 10 MB). Le type MIME et l'extension sont validés côté serveur avant tout traitement. Le fichier est ensuite chargé sur **AWS S3** via le SDK `@aws-sdk/client-s3`. L'URL S3 (clé) est sauvegardée en base. Une entrée dans `document_logs` est créée pour chaque action.

Formats acceptés

PDF · JPG · JPEG · PNG · DOC · DOCX (validation MIME + extension)

Types de documents acceptés

`releve_notes` · `diplome` · `passeport` · `lettre_motivation` · `lettre_recommandation` · `cv` · `justificatif_domicile` · `langue` · `autre`

8.2 Système de PIN de sécurité

Les documents peuvent être protégés par un PIN à 4 chiffres. Le PIN est haché avec `bcryptjs` et stocké dans la colonne `doc_pin_hash` de la table `profiles`. Pour accéder aux documents :

1. `GET /documents/pin/check` — vérifie si un PIN est défini
2. `POST /documents/pin/verify` — soumet le PIN pour déverrouillage
3. En cas d'oubli : `POST /documents/pin/reset/request` envoie un OTP par email (Resend), puis `POST /documents/pin/reset/confirm` crée un nouveau PIN

8.3 Journal d'activité

La table `document_logs` enregistre toutes les actions : upload, suppression, accès, échec PIN. Accessible via `GET /documents/logs`, elle est indexée sur `(user_id, created_at DESC)` pour des lectures rapides.

9. Moteur d'analyse IA

9.1 Architecture de l'analyse

L'analyse IA est le cœur différenciateur de Wekili. Lorsque l'utilisateur déclenche une analyse (`POST /api/analysis/launch`), le backend :

1. Récupère le profil complet de l'utilisateur (profil académique + documents uploadés)
2. Récupère les données enrichies sur les pays cibles (bourses disponibles, taux d'admission, conditions de langue)
3. Construit un prompt structuré intégrant toutes ces données avec des règles anti-hallucination explicites
4. Appelle l'API Anthropic (`@anthropic-ai/sdk`) — modèle Claude
5. Parse la réponse JSON et sauvegarde en base (table `analyses`)
6. Calcule et sauvegarde les scores d'éligibilité dans la table `matches`

9.2 Structure du résultat d'analyse

```
{
  "score_global": 72,           // Score sur 100
  "synthese": "...",           // Résumé du dossier
  "points_forts": ["...", ...], // Forces identifiées
  "axes_amelioration": [...],  // Points à améliorer
  "forces": [...],
  "faiblesses": [...],
  "recommandations": [...],
  "programmes_recommandes": [  // Programmes adaptés au profil
    { "nom": "...", "universite": "...", "pays": "...", "raison": "..." }
  ],
  "estimation_chances": {      // Par pays cible
    "France": { "niveau": "Élevé", "pourcentage": 75, "commentaire": "..." }
  }
}
```

9.3 Lettre de motivation IA

Deux modes disponibles via `/api/lm` :

- **Génération** (`POST /lm/generate`) : le modèle rédige une LM complète adaptée à l'université cible à partir du profil
- **Correction** (`POST /lm/correct`) : l'utilisateur soumet sa propre LM, le modèle l'analyse (score, points forts, améliorations) et propose une version corrigée

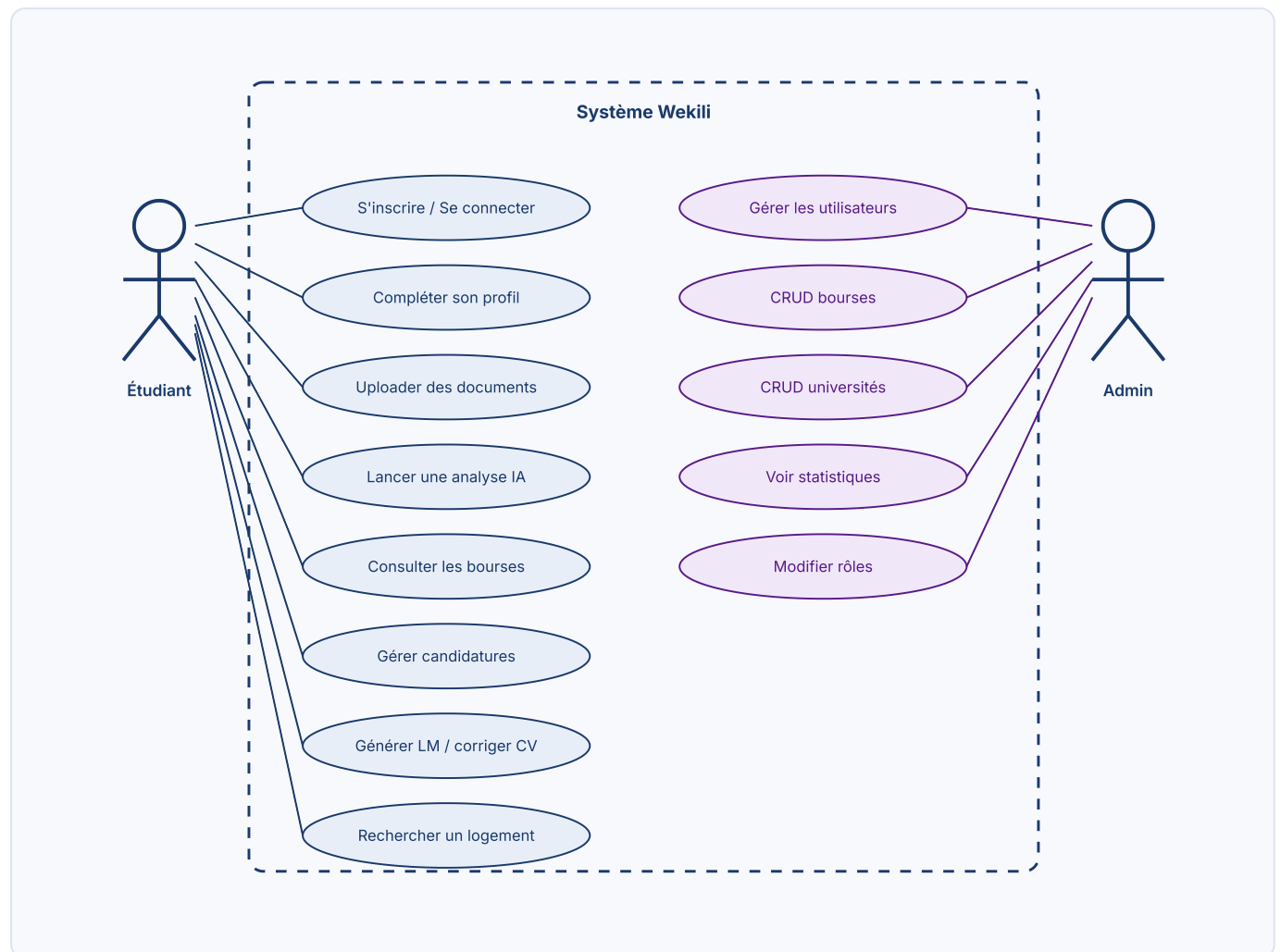
Chaque version est stockée dans `lm_versions` avec versioning automatique.

9.4 Analyse et correction de CV

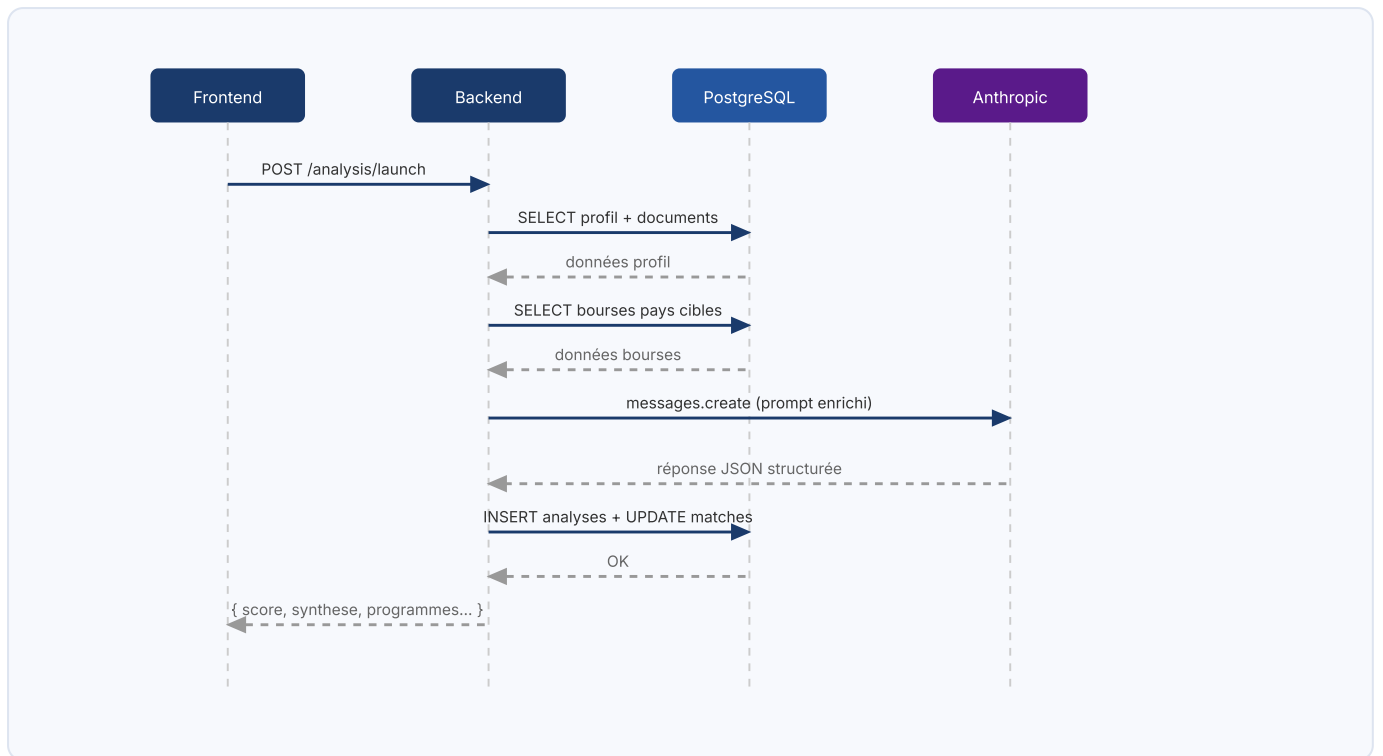
Le CV peut être soumis en texte libre ou extrait d'un PDF via `pdf-parse v2` (classe `PDFParse`, API `new PDFParse({ data: buffer })`). Le modèle IA adapte l'analyse et les corrections aux normes du pays cible (ex : format sans photo pour le Royaume-Uni, format avec photo pour la France). Les résultats incluent un score, les points forts, les corrections prioritaires, les sections manquantes et la norme du pays.

10. Diagrammes UML

10.1 Diagramme de cas d'utilisation



10.2 Diagramme de séquence — Analyse IA



11. Sécurité

Mesures implémentées

Mesure	Implémentation
Headers HTTP	<code>helmet</code> — Content-Security-Policy, X-Frame-Options, HSTS, etc.
CORS	Liste blanche stricte : vercel.app + localhost:5173/3000 uniquement
Rate limiting	200 req/15min global · 10 req/15min sur /auth
Mots de passe	bcryptjs, 10 rounds de salage
Tokens JWT	Signature HMAC-SHA256, expiration configurable
PIN documents	Hash bcrypt stocké en base, jamais en clair
Upload	Validation MIME + extension côté serveur (double vérification)
Rôles (RBAC)	Middleware authAdmin vérifie role en base à chaque requête
2FA	TOTP compatible Google Authenticator
OAuth	Tokens Google vérifiés côté serveur (google-auth-library)
Trust proxy	<code>app.set('trust proxy', 1)</code> pour Render/Railway
Body limit	<code>express.json({ limit: '2mb' })</code>

Vecteurs d'attaque identifiés

Injection SQL — Le driver `pg` utilise des requêtes paramétrées (`$1`, `$2...`) sur toutes les routes contrôleurs. Risque faible si les conventions sont respectées.

XSS — React échappe nativement le JSX. Les données affichées via `{variable}` sont protégées. Aucun `dangerouslySetInnerHTML` détecté.

CSRF — Pas de cookies de session, auth exclusivement par JWT Bearer token. Immunisé CSRF par design.

Exposition S3 — Les URLs S3 ne sont exposées qu'aux utilisateurs authentifiés, via les endpoints API. Les buckets ne doivent pas être publics.

Secrets en variables d'environnement — `AWS_ACCESS_KEY`, `ANTHROPIC_API_KEY`, `RESEND_API_KEY`, `TWILIO_*` stockés en variables d'environnement. Ne jamais committer dans git.

12. Forces et faiblesses

Forces

- **Architecture claire** — Séparation frontend/backend, routes modulaires, contrôleurs isolés
- **Migrations idempotentes** — `setup_all.js` sécurise tous les déploiements avec `IF NOT EXISTS`
- **Multi-auth** — 4 méthodes de connexion (email, Google, téléphone, 2FA) offrent une flexibilité maximale
- **Sécurité documentaire** — PIN bcrypt + OTP email + journal d'activité = triple protection
- **IA contextuelle** — Prompts enrichis par pays avec données bourses réelles et règles anti-hallucination
- **Données structurées** — JSONB PostgreSQL pour critères, avantages, corrections IA → requêtes riches
- **Catalogue complet** — 9 bourses + 15 universités pré-chargées avec critères détaillés
- **UX soignée** — Icônes SVG cohérentes, Tailwind, responsive, animations fluides
- **Versioning LM/CV** — Historique complet des versions avec scores IA
- **RBAC** — Système de rôles user/admin/superadmin complet

Faiblesses

- **Absence de tests** — Aucun test unitaire ni d'intégration. Toute régression est détectée manuellement
- **Pas de refresh token** — JWT sans rotation ; expiration longue = risque de session zombie
- **setup_all.js monolithique** — Toutes les migrations dans un seul fichier (640 lignes). Difficile à maintenir à grande échelle
- **Erreurs générique** — Messages d'erreur HTTP parfois peu informatifs pour l'UX utilisateur
- **S3 sans CDN** — Documents servis directement depuis S3 sans CloudFront → latence variable
- **Pas de pagination côté DB** — Les listes (bourses, universités) chargent tout en mémoire
- **Pas de cache** — Chaque appel analyse, bourses, universités = requête DB fraîche
- **Twilio OTP non stocké** — Dépendance externe totale pour la vérification téléphonique
- **Suppression compte** — La suppression en cascade DB ne supprime pas les fichiers S3
- **Mots de passe oubliés** — Pas d'indication si l'email est inconnu (réponse ambiguë côté UX)

13. Recommandations techniques

Priorité haute

#	Recommandation	Impact
1	Implémenter des tests automatisés — Vitest + React Testing Library pour le frontend, Jest + Supertest pour l'API. Viser 70% de couverture sur les contrôleurs critiques (auth, documents, analysis).	Qualité · Déploiement continu
2	Ajouter un refresh token — JWT access token (15 min) + refresh token httpOnly cookie (7 jours). Réduire la durée du JWT et ajouter une table <code>refresh_tokens</code> .	Sécurité
3	Supprimer les fichiers S3 à la suppression du compte — Lister et supprimer tous les objets S3 préfixés par <code>userId/</code> avant la suppression DB en cascade.	Conformité RGPD · Coûts S3
4	Pagination côté base de données — Ajouter <code>LIMIT / OFFSET</code> sur toutes les listes (bourses, universités, users admin) avec paramètres de page dans les endpoints GET.	Performance · Scalabilité
5	Migrer setup_all.js vers un système de migrations versionné — Adopter <code>node-pg-migrate</code> ou <code>Flyway</code> pour gérer les migrations par numéro de version.	Maintenabilité

Priorité moyenne

#	Recommandation	Impact
6	Ajouter un CDN S3 (CloudFront) — Signer les URLs de documents avec expiration (pre-signed URLs) et servir via CloudFront pour la latence.	Performance · Sécurité
7	Cache Redis pour les données statiques — Mettre en cache le catalogue bourses/universités (TTL 1h) pour éviter les requêtes DB répétées.	Performance
8	Logging structuré — Remplacer les <code>console.log</code> par Winston ou Pino avec niveaux (info/warn/error) et export vers un service (Datadog, Logtail).	Observabilité
9	Variables d'environnement validées au démarrage — Utiliser <code>zod</code> ou <code>envalid</code> pour valider toutes les variables d'environnement requises et stopper le serveur si une manque.	Fiabilité
10	Rate limiting par utilisateur — Ajouter un rate limit sur <code>/api/analysis/launch</code> par <code>userId</code> (ex : 5 analyses/heure) pour contrôler les coûts API Claude.	Coûts · Abus

14. Installation et déploiement

14.1 Prérequis

- Node.js $\geq 18.0.0$
- PostgreSQL 14+ (local ou Neon/Supabase)
- Compte AWS S3 (bucket + IAM avec droits S3:GetObject, S3:PutObject, S3:DeleteObject)
- Clé API Anthropic
- Compte Resend (clé API + domaine vérifié)
- Compte Twilio (Account SID + Auth Token + Verify Service SID)
- Application Google OAuth2 (Client ID)

14.2 Installation locale

```
# Cloner le dépôt
git clone https://github.com/princeahoton/wekili.git
cd wekili

# Backend
cd backend
cp .env.example .env      # remplir les variables
npm install
npm run dev                # nodemon server.js → port 5000

# Frontend (nouveau terminal)
cd ../frontend
npm install
npm run start              # vite → http://localhost:5173
```

14.3 Variables d'environnement backend (.env)

```
DATABASE_URL=postgresql://user:pass@host:5432/wekili
JWT_SECRET=votre-secret-jwt-long-et-aleatoire

# AWS S3
AWS_ACCESS_KEY_ID=AKIAxxxxxxxxxx
AWS_SECRET_ACCESS_KEY=xxxxxxxxxxxxxxxxxxxxxxxxxx
AWS_REGION=eu-west-3
S3_BUCKET_NAME=wekili-documents

# Anthropic
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxxxxx

# Resend (email)
RESEND_API_KEY=re_xxxxxxxxxxxxxxxxxx
RESEND_FROM=noreply@votredomaine.com

# Twilio (SMS OTP)
```

```

TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=xxxxxxxxxxxxxxxx
TWILIO_VERIFY_SERVICE_SID=VAxxxxxxxxxxxxxxxx

# Google OAuth
GOOGLE_CLIENT_ID=xxxxxxxx.apps.googleusercontent.com

PORT=5000

```

14.4 Déploiement production

Backend — Render

1. Créer un Web Service sur Render, connecter le dépôt GitHub
2. Build Command : `npm install`
3. Start Command : `node server.js`
4. Ajouter toutes les variables d'environnement dans l'interface Render
5. Les migrations s'exécutent automatiquement au premier démarrage via `runAll()`

Frontend — Vercel

1. Importer le projet depuis GitHub dans Vercel
2. Définir le Root Directory sur `frontend/`
3. Build Command : `npm run build` · Output : `dist/`
4. Ajouter la variable d'environnement `VITE_API_URL=https://votre-backend.onrender.com`

Premier administrateur

```

-- Après inscription, promouvoir le compte admin via SQL :
UPDATE users SET role = 'admin'
WHERE email = 'votre-email-admin@example.com';

```

15. Guide de maintenance

15.1 Ajout d'une bourse

Via l'interface admin (`/admin/bourses`) ou directement en base :

```
INSERT INTO scholarships (nom, organisme, pays, code_pays, niveau, montant,
description, lien, deadline, langue_requise, niveau_langue_requis, type_financement)
VALUES ('Nom bourse', 'Organisme', 'France', 'fr', 'Master', '1 000 €/mois',
'Description', 'https://lien.com', '2027-01-31', 'Français', 'B2', 'Bourse complète');
```

15.2 Ajout d'une université

Via l'interface admin (`/admin/universities`). Les colonnes essentielles sont : `nom` , `pays` , `code_pays` , `ville` , `type` , `domaines` (TEXT[]), `niveaux` (TEXT[]), `taux_admission` , `classement_mondial` , `date_ouverture` , `date_cloture` .

15.3 Mise à jour du modèle IA

Le modèle Claude utilisé est configuré dans `backend/controllers/analysisController.js` . Pour passer à une version plus récente, modifier le paramètre `model` dans l'appel `anthropic.messages.create()` . Tester les prompts en environnement de staging avant de passer en production.

15.4 Monitoring des coûts

Service	Point de contrôle	Fréquence
Anthropic	Console Anthropic → Usage dashboard	Hebdomadaire
AWS S3	AWS Cost Explorer → S3	Mensuelle
Resend	Dashboard Resend → Emails envoyés	Mensuelle
Twilio	Console Twilio → Usage	Mensuelle
Render	Dashboard Render → Metrics	Hebdomadaire

15.5 Sauvegardes

- **Base de données** : activer les sauvegardes automatiques sur Neon/Supabase (rétention 7 jours recommandée)
- **S3** : activer la versioning S3 sur le bucket pour récupérer les fichiers supprimés accidentellement
- **Code** : dépôt GitHub avec branches protégées sur `main`

15.6 Procédure de mise à jour

1. Créer une branche feature depuis `main`
2. Développer et tester localement
3. Pull Request → revue de code
4. Merge sur `main` → déploiement automatique Vercel + Render
5. Vérifier les logs Render au démarrage (migrations) et les métriques Vercel

15.7 Diagnostics fréquents

Symptôme	Cause probable	Action
Erreur 500 au démarrage backend	Migration échouée	Vérifier les logs Render, connexion DB, variables d'environnement
Upload documents échoue	Variables AWS incorrectes ou bucket inaccessible	Tester la connexion S3 avec aws-sdk en script isolé
Analyse IA timeout	Contexte prompt trop long ou quota Anthropic dépassé	Vérifier l'usage Anthropic, réduire le contexte si nécessaire
Emails non reçus	Domaine Resend non vérifié ou quota dépassé	Vérifier les logs Resend, statut DNS du domaine
Connexion Google échoue	Google Client ID incorrect ou origin non autorisée	Vérifier la console Google Cloud → Credentials → Authorized origins